



(21) 申请号 202610486180.1

(22) 申请日 2026.04.14

(71) 申请人 上海燧原科技股份有限公司

地址 201306 上海市浦东新区中国(上海)
自由贸易试验区临港新片区业盛路
188号A-522室

(72) 发明人 周荣华 周叶飞

(74) 专利代理机构 北京品源专利代理有限公司
11332

专利代理师 侯军洋

(51) Int. Cl.

G06F 11/362 (2025.01)

G06F 8/41 (2018.01)

G06F 9/445 (2018.01)

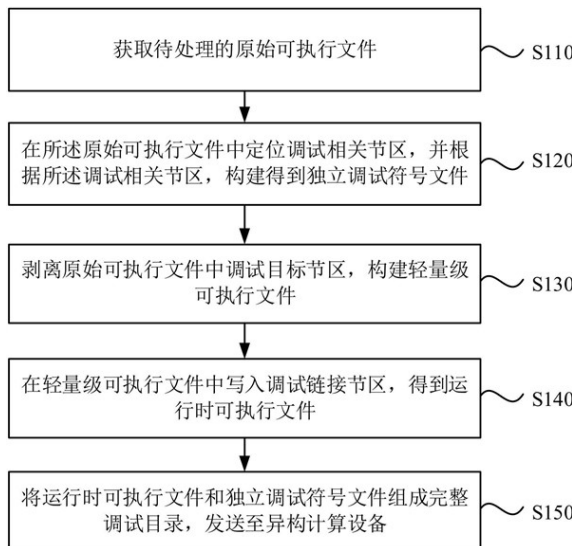
权利要求书3页 说明书22页 附图7页

(54) 发明名称

一种异构计算胖二进制调试符号与运行时
文件分离的方法

(57) 摘要

本发明公开了一种异构计算胖二进制调试符号与运行时文件分离的方法,涉及异构计算技术领域,该方法包括:通过编译处理设备将原始可执行文件中的调试相关节区提取并构建独立调试符号文件,剥离原始可执行文件中调试目标节区,形成轻量级可执行文件,在文件中写入用于建立关联映射的调试链接节区,使得异构计算设备在执行运行时可执行文件的过程中响应调试请求时,基于该调试链接节区关联加载对应的独立调试符号文件,保证了调试操作的正常执行,降低了调试过程中的数据传输成本,将运行时可执行文件与独立调试符号文件组成完整调试目录发送,可确保二者的对应性,提升调试准确性和效率,适配异构计算设备的轻量化部署与高效调试需求。



1. 一种异构计算胖二进制调试符号与运行时文件分离的方法,其特征在于,由编译处理设备执行,包括:

获取待处理的原始可执行文件;

在所述原始可执行文件中定位调试相关节区,并根据所述调试相关节区,构建得到独立调试符号文件;

剥离原始可执行文件中调试目标节区,构建轻量级可执行文件;其中,调试相关节区的内容至少包含调试目标节区的内容;

在轻量级可执行文件中写入调试链接节区,得到运行时可执行文件;其中,所述调试链接节区用于使运行时可执行文件与独立调试符号文件建立关联映射;

将运行时可执行文件和独立调试符号文件组成完整调试目录,发送至异构计算设备;

其中,所述运行时可执行文件用于在异构计算设备接收后加载至内存并执行,且在执行过程中响应调试请求时,基于所述调试链接节区关联加载对应的独立调试符号文件,以执行与调试请求匹配的调试操作。

2. 根据权利要求1所述方法,其特征在于,在获取待处理的原始可执行文件之前,还包括:

向链接命令中注入构建唯一标识生成参数,通过执行链接命令编译链接胖二进制格式的原始可执行文件,并在原始可执行文件中写入构建唯一标识节区,得到待处理的原始可执行文件;

其中,所述调试链接节区存储独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识;所述构建唯一标识用于唯一标识原始可执行文件的构建版本,使运行时可执行文件与独立调试符号文件保持版本一致。

3. 根据权利要求1所述方法,其特征在于,所述调试相关节区包括以调试标识为前缀命名的节区、用于存储静态符号信息的静态符号表节区和用于存储静态字符串信息的静态字符串表节区;所述调试链接节区存储独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识。

4. 根据权利要求3所述方法,其特征在于,调试目标节区包括:以调试标识为前缀命名的节区;

相应的,剥离原始可执行文件中调试目标节区,构建轻量级可执行文件,包括:

剥离原始可执行文件中调试目标节区,保留用于支撑异构计算设备端代码运行的核心数据节区,得到轻量级可执行文件;

其中,所述核心数据节区包括用于存储可执行指令的代码段、用于存储通用注释信息的通用注释节区、用于存储动态符号信息的动态符号表、用于存储哈希索引的哈希表节区、用于存储动态字符串信息的动态字符串表、用于存储动态链接指令的动态链接信息、用于存储未初始化数据的未初始化数据段、用于存储静态符号信息的调试相关节区中的静态符号表节区、用于存储静态字符串信息的调试相关节区中的静态字符串表节区、用于存储编译器生成注释的编译器注释节区、用于存储节区名称映射的节区名称表节区、用于存储内核识别描述信息的内核描述节区、以及用于存储构建唯一标识的构建唯一标识节区。

5. 一种异构计算胖二进制调试符号与运行时文件分离的方法,其特征在于,由异构计算设备执行,该方法包括:

接收编译处理设备发送的完整调试目录,并将完整调试目录中的运行时可执行文件加载至内存中执行;

其中,所述运行时可执行文件为剥离掉原始可执行文件中的调试目标节区后得到的轻量级可执行文件,在所述运行时可执行文件中包括使运行时可执行文件与独立调试符号文件建立关联映射的调试链接节区;

在执行内存中的运行时可执行文件的过程中,响应于调试请求,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作。

6. 根据权利要求5所述方法,其特征在于,将完整调试目录中的运行时可执行文件加载至内存中执行,包括:

读取运行时可执行文件中核心数据节区,并将核心数据节区加载至内存中,执行内存中核心数据节区中的内容。

7. 根据权利要求5所述方法,其特征在于,所述调试链接节区存储独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识;

相应的,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作,包括:

读取运行时可执行文件,解析所述运行时可执行文件的调试链接节区,得到调试关联信息;其中,调试关联信息包括引用标识、循环冗余校验值及构建唯一标识;

根据调试关联信息,在与引用标识匹配的搜索路径中,搜索与运行时可执行文件匹配的独立调试符号文件;

若搜索到匹配的独立调试符号文件,则基于循环冗余校验值和构建唯一标识对独立调试符号文件进行校验,在校验通过后将该独立调试符号文件加载至内存,并将独立调试符号文件与内存中匹配的运行时可执行文件建立关联;

基于运行时可执行文件和关联的独立调试符号文件,执行与调试请求匹配的调试操作。

8. 根据权利要求7所述方法,其特征在于,在搜索与运行时可执行文件匹配的独立调试符号文件之后,还包括:

若未搜索到匹配的独立调试符号文件,或者基于循环冗余校验值对独立调试符号文件的校验未通过,或者基于构建唯一标识对独立调试符号文件的一致性验证未通过,则基于运行时可执行文件中的调试相关节区执行与调试请求匹配的调试操作。

9. 一种异构计算胖二进制调试符号与运行时文件分离的装置,其特征在于,配置于编译处理设备,所述装置包括:

文件获取模块,用于获取待处理的原始可执行文件;

独立文件模块,用于在所述原始可执行文件中定位调试相关节区,并根据所述调试相关节区,构建得到独立调试符号文件;

轻量文件模块,用于剥离原始可执行文件中调试目标节区,构建轻量级可执行文件;其中,调试相关节区的内容至少包含调试目标节区的内容;

写入节区模块,用于在轻量级可执行文件中写入调试链接节区,得到运行时可执行文件;其中,所述调试链接节区用于使运行时可执行文件与独立调试符号文件建立关联映射;

目录发送模块,用于将运行时可执行文件和独立调试符号文件组成完整调试目录,发送至异构计算设备;

其中,所述运行时可执行文件用于在异构计算设备接收后加载至内存并执行,且在执行过程中响应调试请求时,基于所述调试链接节区关联加载对应的独立调试符号文件,以执行与调试请求匹配的调试操作。

10.一种异构计算胖二进制调试符号与运行时文件分离的装置,其特征在于,配置于异构计算设备,所述装置包括:

目录接收模块,用于接收编译处理设备发送的完整调试目录,并将完整调试目录中的运行时可执行文件加载至内存中执行;

其中,所述运行时可执行文件为剥离掉原始可执行文件中的调试目标节区后得到的轻量级可执行文件,在所述运行时可执行文件中包括使运行时可执行文件与独立调试符号文件建立关联映射的调试链接节区;

调试响应模块,用于在执行内存中的运行时可执行文件的过程中,响应于调试请求,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作。

11.一种编译处理设备,其特征在于,所述编译处理设备包括:

至少一个处理器;以及与所述至少一个处理器通信连接的存储器;其中,所述存储器存储有可被所述至少一个处理器执行的计算机程序,所述计算机程序被所述至少一个处理器执行,以使所述至少一个处理器能够执行权利要求1-4任一所述的异构计算胖二进制调试符号与运行时文件分离的方法。

12.一种异构计算设备,其特征在于,所述异构计算设备包括:

至少一个处理器;以及与所述至少一个处理器通信连接的存储器;其中,所述存储器存储有可被所述至少一个处理器执行的计算机程序,所述计算机程序被所述至少一个处理器执行,以使所述至少一个处理器能够执行权利要求5-8任一所述的异构计算胖二进制调试符号与运行时文件分离的方法。

13.一种计算机可读存储介质,其特征在于,所述计算机可读存储介质存储有计算机指令,所述计算机指令用于使处理器执行时实现权利要求1-8任一所述的异构计算胖二进制调试符号与运行时文件分离的方法。

14.一种计算机程序产品,其特征在于,所述计算机程序产品包括计算机程序,所述计算机程序在被处理器执行时实现根据权利要求1-8任一所述的异构计算胖二进制调试符号与运行时文件分离的方法。

一种异构计算胖二进制调试符号与运行时文件分离的方法

技术领域

[0001] 本发明涉及异构计算技术领域,尤其涉及一种异构计算胖二进制调试符号与运行时文件分离的方法。

背景技术

[0002] 在GPU(Graphics Processing Unit,图形处理器)、NPU(Neural Processing Unit,神经网络处理器)、XPU(X Processing Unit,X处理器,各类加速芯片的统称)等异构计算场景中,设备端程序与算子库多采用胖二进制(fatbin)格式进行分发,其内部集成了适配多种底层芯片架构的可执行文件。

[0003] 当前异构计算领域普遍采用携带完整调试信息的胖二进制文件进行程序分发与部署,文件中同时包含运行所需的代码段、数据段以及各类调试节区。

[0004] 然而,编译生成的胖二进制文件中,调试相关节区在整体文件中占据极高比例,实际参与运行的代码段与数据段仅占较小比重,由于调试信息与运行时代码耦合存储,一方面导致生产环境部署的文件体积显著偏大,大幅提升网络传输与存储开销,同时降低文件加载效率,影响设备端运行性能;另一方面,若直接采用常规方式剥离调试信息,虽能减小文件体积,但会使调试器失去必要符号支撑,无法实现断点调试、堆栈回溯等关键调试功能,且运行时文件与调试信息之间缺少可靠的关联机制,难以在轻量化部署与完整调试能力之间实现兼顾。

发明内容

[0005] 本发明提供了一种异构计算胖二进制调试符号与运行时文件分离的方法,以解决异构计算时网络传输与存储开销大、文件加载效率低、以及难以满足轻量化部署与调试需求的问题。

[0006] 根据本发明实施例的一方面,提供了一种异构计算胖二进制调试符号与运行时文件分离的方法,由编译处理设备执行,包括:

获取待处理的原始可执行文件;在所述原始可执行文件中定位调试相关节区,并根据所述调试相关节区,构建得到独立调试符号文件;剥离原始可执行文件中调试目标节区,构建轻量级可执行文件;其中,调试相关节区的内容至少包含调试目标节区的内容;在轻量级可执行文件中写入调试链接节区,得到运行时可执行文件;其中,所述调试链接节区用于使运行时可执行文件与独立调试符号文件建立关联映射;将运行时可执行文件和独立调试符号文件组成完整调试目录,发送至异构计算设备;其中,所述运行时可执行文件用于在异构计算设备接收后加载至内存并执行,且在执行过程中响应调试请求时,基于所述调试链接节区关联加载对应的独立调试符号文件,以执行与调试请求匹配的调试操作。

[0007] 根据本发明实施例的另一方面,提供了一种异构计算胖二进制调试符号与运行时文件分离的方法,由异构计算设备执行,该方法包括:

接收编译处理设备发送的完整调试目录,并将完整调试目录中的运行时可执行文

件加载至内存中执行;其中,所述运行时可执行文件为剥离掉原始可执行文件中的调试目标节区后得到的轻量级可执行文件,在所述运行时可执行文件中包括使运行时可执行文件与独立调试符号文件建立关联映射的调试链接节区;在执行内存中的运行时可执行文件的过程中,响应于调试请求,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作。

[0008] 根据本发明实施例的另一方面,提供了一种异构计算胖二进制调试符号与运行时文件分离的装置,配置于编译处理设备,所述装置包括:

文件获取模块,用于获取待处理的原始可执行文件;独立文件模块,用于在所述原始可执行文件中定位调试相关节区,并根据所述调试相关节区,构建得到独立调试符号文件;轻量文件模块,用于剥离原始可执行文件中调试目标节区,构建轻量级可执行文件;其中,调试相关节区的内容至少包含调试目标节区的内容;写入节区模块,用于在轻量级可执行文件中写入调试链接节区,得到运行时可执行文件;其中,所述调试链接节区用于使运行时可执行文件与独立调试符号文件建立关联映射;目录发送模块,用于将运行时可执行文件和独立调试符号文件组成完整调试目录,发送至异构计算设备;其中,所述运行时可执行文件用于在异构计算设备接收后加载至内存并执行,且在执行过程中响应调试请求时,基于所述调试链接节区关联加载对应的独立调试符号文件,以执行与调试请求匹配的调试操作。

[0009] 根据本发明实施例的另一方面,提供了一种异构计算胖二进制调试符号与运行时文件分离的装置,配置于异构计算设备,所述装置包括:

目录接收模块,用于接收编译处理设备发送的完整调试目录,并将完整调试目录中的运行时可执行文件加载至内存中执行;其中,所述运行时可执行文件为剥离掉原始可执行文件中的调试目标节区后得到的轻量级可执行文件,在所述运行时可执行文件中包括使运行时可执行文件与独立调试符号文件建立关联映射的调试链接节区;调试响应模块,用于在执行内存中的运行时可执行文件的过程中,响应于调试请求,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作。

[0010] 根据本发明实施例的另一方面,提供了一种编译处理设备,所述编译处理设备包括:

至少一个处理器;以及与所述至少一个处理器通信连接的存储器;其中,所述存储器存储有可被所述至少一个处理器执行的计算机程序,所述计算机程序被所述至少一个处理器执行,以使所述至少一个处理器能够执行本发明任一实施例所述的由编译处理设备执行的异构计算胖二进制调试符号与运行时文件分离的方法。

[0011] 根据本发明实施例的另一方面,提供了一种异构计算设备,所述异构计算设备包括:

至少一个处理器;以及与所述至少一个处理器通信连接的存储器;其中,所述存储器存储有可被所述至少一个处理器执行的计算机程序,所述计算机程序被所述至少一个处理器执行,以使所述至少一个处理器能够执行本发明任一实施例所述的由异构计算设备执行的异构计算胖二进制调试符号与运行时文件分离的方法。

[0012] 根据本发明实施例的另一方面,提供了一种计算机可读存储介质,所述计算机可

读存储介质存储有计算机指令,所述计算机指令用于使处理器执行时实现本发明任一实施例所述的异构计算胖二进制调试符号与运行时文件分离的方法。

[0013] 根据本发明实施例的另一方面,还提供了一种计算机程序产品,包括计算机程序,所述计算机程序被处理器执行时实现如本发明任一实施例所述方法的步骤。

[0014] 本发明实施例的技术方案,通过编译处理设备将原始可执行文件中的调试相关节区提取并构建独立调试符号文件,剥离原始可执行文件中无需参与运行的调试目标节区,形成轻量级的运行时可执行文件,减小运行时可执行文件的体积,降低该文件在异构计算设备与编译处理设备之间的传输带宽占用,减少异构计算设备的存储资源消耗,且轻量级的运行时可执行文件能加快异构计算设备加载文件的速度,提升程序运行效率;在运行时可执行文件中写入用于建立关联映射的调试链接节区,使得异构计算设备在执行运行时可执行文件的过程中响应调试请求时,基于该调试链接节区关联加载对应的独立调试符号文件,无需重新传输完整的带调试符号的原始可执行文件,保证了调试操作的正常执行,实现调试能力的完整保留,降低了调试过程中的数据传输成本,将运行时可执行文件与独立调试符号文件组成完整调试目录发送,可确保二者的对应性,避免调试符号与运行时文件不匹配的问题,提升调试准确性和效率,整体适配GPU、NPU、XPU等异构计算设备的轻量化部署与高效调试需求,兼顾了程序运行性能与调试便捷性。

[0015] 应当理解,本部分所描述的内容并非旨在标识本发明的实施例的关键或重要特征,也不用于限制本发明的范围。本发明的其它特征将通过以下的说明书而变得容易理解。

附图说明

[0016] 为了更清楚地说明本发明实施例中的技术方案,下面将对实施例描述中所需要使用的附图作简单地介绍,显而易见地,下面描述中的附图仅仅是本发明的一些实施例,对于本领域普通技术人员来讲,在不付出创造性劳动的前提下,还可以根据这些附图获得其他的附图。

[0017] 图1是根据本发明实施例一提供的一种异构计算胖二进制调试符号与运行时文件分离的方法的流程图;

图2是本发明实施例适用的一种调试符号分离流程示意图;

图3是本发明实施例适用的一种可执行文件结构对比示意图;

图4是根据本发明实施例二提供的另一种异构计算胖二进制调试符号与运行时文件分离的方法的流程图;

图5是本发明实施例适用的一种运行时兼容性加载示意图;

图6是本发明实施例适用的一种调试器加载示意图;

图7是根据本发明实施例三提供的又一种异构计算胖二进制调试符号与运行时文件分离的方法的流程图;

图8是本发明实施例适用的一种异构计算胖二进制调试符号与运行时文件分离示意图;

图9是根据本发明实施例四提供的一种异构计算胖二进制调试符号与运行时文件分离的装置的结构示意图;

图10是根据本发明实施例五提供的另一种异构计算胖二进制调试符号与运行时

文件分离的装置的结构示意图；

图11是实现本发明实施例的异构计算胖二进制调试符号与运行时文件分离的方法的编译处理设备的结构示意图；

图12是实现本发明实施例的异构计算胖二进制调试符号与运行时文件分离的方法的异构计算设备的结构示意图。

具体实施方式

[0018] 为了使本技术领域的人员更好地理解本发明方案，下面将结合本发明实施例中的附图，对本发明实施例中的技术方案进行清楚、完整地描述，显然，所描述的实施例仅仅是本发明一部分的实施例，而不是全部的实施例。基于本发明中的实施例，本领域普通技术人员在没有做出创造性劳动前提下所获得的所有其他实施例，都应当属于本发明保护的范围。

[0019] 需要说明的是，本发明的说明书和权利要求书及上述附图中的术语“第一”、“第二”等是用于区别类似的对象，而不必用于描述特定的顺序或先后次序。应该理解这样使用的数据在适当情况下可以互换，以便这里描述的本发明的实施例能够以除了在这里图示或描述的那些以外的顺序实施。此外，术语“包括”和“具有”及其任何变形，意图在于覆盖不排他的包含，例如，包含了一系列步骤或单元的过程、方法、系统、产品或设备不必限于清楚地列出的那些步骤或单元，而是可包括没有清楚地列出的或对于这些过程、方法、产品或设备固有的其它步骤或单元。

[0020] 实施例一

图1为本发明实施例一提供的一种异构计算胖二进制调试符号与运行时文件分离的方法的流程图，本实施例可适用于分离异构计算中胖二进制调试符号与运行时文件的情况，该方法可以由异构计算胖二进制调试符号与运行时文件分离的装置来执行，该异构计算胖二进制调试符号与运行时文件分离的装置可以采用硬件和/或软件的形式实现，并一般可配置于编译处理设备中。

[0021] 本发明实施例中，编译处理设备具体可以理解为：用于执行编译、以及调试符号分离等操作的电子设备，是基于统一的操作系统或社区标准（如GNU（GNU's Not Unix，GNU不是Unix）相关标准）完成调试符号提取、以及文件生成等操作，可采用GPU、NPU、或XPU等硬件形态。

[0022] 操作系统或社区标准具体可以理解为：编译处理设备与异构计算设备遵循的统一运行规范，确保两者的数据传输、文件识别、调试操作等保持兼容，无需额外适配。GNU标准具体可以理解为：通用的程序编译、调试、链接相关规范，适配异构计算场景，是编译处理设备和异构计算设备实现文件分离和调试的依据。

[0023] 异构计算设备具体可以理解为：用于接收编译处理设备发送的文件并执行程序加载、运行及调试操作的电子设备，可采用GPU、NPU或XPU等硬件形态，无需额外修改即可适配分离后的文件。

[0024] 如图1所示，该方法包括：

S110、获取待处理的原始可执行文件。

[0025] 本发明实施例中，原始可执行文件具体可以理解为：编译生成的包含完整代码与

调试信息的胖二进制可执行与可链接格式(Executable and Linkable Format,ELF)的文件,是后续处理的对象。

[0026] 进一步的,在上述各实施例的基础上,在获取待处理的原始可执行文件之前,还可以包括:

向链接命令中注入构建唯一标识生成参数,通过执行链接命令编译链接胖二进制格式的原始可执行文件,并在原始可执行文件中写入构建唯一标识节区,得到待处理的原始可执行文件;其中,所述调试链接节区存储独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识;所述构建唯一标识用于唯一标识原始可执行文件的构建版本,使运行时可执行文件与独立调试符号文件保持版本一致。

[0027] 本发明实施例中,构建唯一标识生成参数具体可以理解为:用于在编译链接阶段生成文件唯一标识的参数,例如--build-id,也可以为其他形式.--build-id为GNU编译器与链接器的命令行参数,用于在链接阶段为原始ELF文件自动生成一串全局唯一的标识,是启用构建唯一标识功能的关键配置项。

[0028] 构建唯一标识具体可以理解为:用于唯一标记一次编译构建版本,保证运行文件与调试符号版本匹配的标识,例如Build-ID。Build-ID是一串由系统生成的哈希值,用于唯一标记某一次编译构建生成的可执行文件。

[0029] 构建唯一标识节区具体可以理解为:存放构建唯一标识生成参数的专用节区,例如.note.gnu.build-id。note.gnu.build-id为ELF文件中用于存储Build-ID的标准节区名称,遵循GNU规范,属于固定命名的系统节区,用于存放版本唯一标识。

[0030] 调试链接节区具体可以理解为:用于记录运行文件与调试符号关联信息的专用节区。循环冗余校验值具体可以理解为:用于校验调试符号文件完整性的参数。引用标识具体可以理解为:用于定位独立调试符号文件的标识信息。

[0031] 具体的,在获取待处理的原始可执行文件前,除了直接使用现有文件外,还可向链接命令中自动注入用于生成构建唯一标识的参数(如--build-id参数),执行链接命令编译得到胖二进制格式的原始可执行文件,并在文件中写入专门存放构建唯一标识的节区(如.note.gnu.build-id)以存储构建唯一标识,得到后续处理所需的原始可执行文件,除了使用--build-id方式外,也可通过自定义编译脚本或链接脚本生成唯一版本标识并写入对应节区。

[0032] 后续在调试链接节区中同时存储引用标识、循环冗余校验值和构建唯一标识(如Build-ID),利用构建唯一标识实现运行时可执行文件与独立调试符号文件的版本绑定,保证调试过程中二者严格匹配。

[0033] 通过在链接阶段向链接命令注入构建唯一标识生成参数,能够在胖二进制格式的原始可执行文件中生成并写入专属的构建唯一标识节区,使得每个构建版本都具备唯一的身份标识,进而将该构建唯一标识与引用标识、循环冗余校验值一同存储在调试链接节区中,能够在调试过程中精准校验运行时可执行文件与独立调试符号文件的版本对应关系,从根源上避免因版本不匹配、文件错乱导致的调试失效、定位偏差等问题,同时依托唯一且稳定的版本标识,提升了调试符号匹配的可靠性与准确性,保障异构计算场景下调试流程的稳定执行。

[0034] S120、在所述原始可执行文件中定位调试相关节区,并根据所述调试相关节区,构

建得到独立调试符号文件。

[0035] 本发明实施例中,调试相关节区具体可以理解为:文件中用于承载调试信息的节区。独立调试符号文件具体可以理解为:从原始可执行文件中单独提取出来的专门存储调试信息的文件。

[0036] 具体的,获取需要处理的胖二进制原始可执行文件,在文件内部查找并定位所有与调试功能相关的节区,例如包含debug关键词的相关节区及带有可选标记的相关代码段与数据段,将这些定位到的调试相关节区提取出来,单独组织生成一个只包含调试信息的独立调试符号文件,例如可以采用GNU工具链中objcopy工具配合--only-keep-debug参数提取调试信息,也可通过自定义节区解析程序读取对应节区数据并封装为独立文件。相应的,可以按照指定命名规则生成以“.debug”为后缀的独立调试符号文件,实现调试信息完整提取并与运行时文件分离。

[0037] 其中,objcopy具体可以理解为:GNU工具链中用于操作ELF文件的工具,--only-keep-debug是其参数,作用为仅保留调试相关内容并输出为独立文件。

[0038] 可选的,在上述各实施例的基础上,调试相关节区可以包括以调试标识为前缀命名的节区、用于存储静态符号信息的静态符号表节区和用于存储静态字符串信息的静态字符串表节区;所述调试链接节区存储独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识。

[0039] 本发明实施例中,调试标识为前缀命名的节区具体可以理解为:专门存放调试信息的以调试标识开头的所有节区,例如以“.debug_”开头的所有节区。静态符号表节区具体可以理解为:存放函数名和变量名等静态符号信息的节区,例如以“.symtab”为后缀的节区。静态字符串表节区具体可以理解为:存放符号名称和字符串常量的节区,例如以“.strtab”为后缀的节区。

[0040] 静态符号表具体可以理解为:ELF文件里存放所有函数、变量、全局符号和局部符号的完整列表,包含调试、链接、运行用到的全部符号信息,体积大,主要供调试和链接使用。静态字符串表具体可以理解为:和静态符号表配套使用,存放静态符号表里所有符号对应的名称字符串(如函数名、变量名、文件名)。

[0041] 具体的,在原始可执行文件中定位包含调试前缀节区、静态符号表节区和静态字符串表节区在内的调试相关节区,将这些节区的信息提取出来构建独立调试符号文件,并在调试链接节区中存入引用标识、循环冗余校验值和构建唯一标识,实现调试信息的完整提取与关联标记。

[0042] 在一个具体的例子中,在原始可执行文件中定位调试相关节区,例如以“.debug_”为前缀的节区、以“.symtab”为后缀的静态符号表节区和以“.strtab”为后缀的静态字符串表节区。

[0043] 其中,以“.debug_”为前缀的节区具体可以包括:.debug_str、.debug_loc、.debug_abbrev、.debug_info、.debug_ranges、.debug_frame、以及.debug_line等各类标准调试节区。

[0044] .debug_str节区具体可以理解为:存储调试过程中所需的字符串常量,如变量名、函数名、以及类型名称等,是调试信息的字符存储载体。.debug_loc节区具体可以理解为:记录变量在内存中的位置信息及其生命周期范围,用于调试器在程序运行的不同阶段准确

获取变量值。

[0045] `.debug_abbrev`节区具体可以理解为:定义`.debug_info`节区中各调试信息条目的编码格式和缩写,用于压缩调试信息体积并规范数据结构解析规则。`.debug_info`节区具体可以理解为:存储调试语义信息,包括类型定义、函数声明、以及变量描述等,是调试符号解析的依据。

[0046] `.debug_ranges`节区具体可以理解为:记录代码地址区间与调试信息的对应关系,明确不同代码段对应的调试元数据范围,辅助调试器定位代码关联信息。`.debug_frame`节区具体可以理解为:存储程序调用栈帧的相关调试信息,包括栈帧结构、以及寄存器保存位置等,支持调试过程中的堆栈回溯操作。`.debug_line`节区具体可以理解为:记录源代码文件路径、行号与机器代码地址之间的映射关系,是实现源代码级调试(断点设置和行号定位等)的基础。

[0047] 除使用`objcopy --only-keep-debug`工具提取`.debug_*`及可选静态符号表、静态字符串表外,还可通过自定义解析程序读取并抽取所有调试相关数据,构建得到独立调试符号文件,同时调试链接节区存储文件名、地址形式、或路径索引等独立调试符号文件定位信息的引用标识,并配合存储循环冗余校验值和构建唯一标识,实现调试符号的精准提取、可靠关联与高效匹配。

[0048] 通过将调试相关节区限定为包含调试标识前缀的节区、静态符号表节区以及静态字符串表节区,能够完整覆盖程序调试所需的全部信息,确保提取生成的独立调试符号文件具备完整、可靠的调试能力,同时在调试链接节区中同步存储独立调试符号文件的引用标识、循环冗余校验值和构建唯一标识,既可以通过引用标识实现运行时可执行文件与调试符号文件的快速精准匹配,又能够通过循环冗余校验值保障调试符号文件的完整性与未篡改状态,还可以借助构建唯一标识确保运行时文件与调试符号文件严格属于同一构建版本,从而全面提升调试符号关联的准确性、稳定性和安全性,避免版本不匹配、文件损坏、符号丢失等问题,保障异构计算场景下调试操作的顺利执行。

[0049] `S130`、剥离原始可执行文件中调试目标节区,构建轻量级可执行文件。

[0050] 其中,调试相关节区的内容至少包含调试目标节区的内容。

[0051] 本发明实施例中,调试目标节区具体可以理解为:用于调试且不参与程序运行的节区,是调试相关节区的一部分。轻量级可执行文件具体可以理解为:删除调试目标节区后体积更小、加载更快的运行文件。

[0052] 具体的,从原始可执行文件中移除专门用于调试的调试目标节区,保留程序正常运行所需的内容,例如通过`objcopy --strip-debug`工具命令直接剥离调试信息,或通过自定义节区处理程序删除调试目标节区,以此构建出体积更小的轻量级可执行文件。其中调试相关节区所包含的信息范围完全覆盖调试目标节区,确保剥离操作能够精准移除调试信息且不影响程序运行。其中,`objcopy --strip-debug`具体可以理解为:一种GNU工具链命令,用于剥离调试信息。

[0053] 可选的,在上述各实施例的基础上,调试目标节区可以包括:以调试标识为前缀命名的节区;相应的,在上述各实施例的基础上,剥离原始可执行文件中调试目标节区,构建轻量级可执行文件,可以包括:

剥离原始可执行文件中调试目标节区,保留用于支撑异构计算设备端代码运行的

核心数据节区,得到轻量级可执行文件;其中,所述核心数据节区包括用于存储可执行指令的代码段、用于存储通用注释信息的通用注释节区、用于存储动态符号信息的动态符号表、用于存储哈希索引的哈希表节区、用于存储动态字符串信息的动态字符串表、用于存储动态链接指令的动态链接信息、用于存储未初始化数据的未初始化数据段、用于存储静态符号信息的调试相关节区中的静态符号表节区、用于存储静态字符串信息的调试相关节区中的静态字符串表节区、用于存储编译器生成注释的编译器注释节区、用于存储节区名称映射的节区名称表节区、用于存储内核识别描述信息的内核描述节区、以及用于存储构建唯一标识的构建唯一标识节区。

[0054] 本发明实施例中,核心数据节区具体可以理解为:程序正常运行必须依赖的所有节区,剥离后完整保留,保障程序执行。代码段具体可以理解为:存放可执行指令的节区。动态符号表具体可以理解为:ELF文件中用于程序运行时动态加载、动态链接所需的符号表,只保存运行时必须用到的函数、变量符号,是程序正常执行不可缺少的信息。动态字符串表具体可以理解为:与动态符号表配对使用,存储动态符号表里符号对应的名称字符串(如函数名、变量名、库名),为动态符号表提供字符串支撑。

[0055] 具体的,剥离原始可执行文件中以调试标识为前缀命名的调试目标节区,并完整保留支撑程序正常运行所需的代码段、各类符号表、链接信息、数据段、注释信息、内核描述以及构建唯一标识等核心数据节区,从而生成体积更小、加载更快的轻量级可执行文件,在删除冗余调试信息的同时确保程序运行稳定、功能完整。

[0056] 在一个具体的例子中,表1为原始可执行文件中节区结构。如表1所示,.text节区类型为程序位(PROGBITS)类,是存储可执行指令的代码段,分离后完整保留;.note节区类型为注释(NOTE)类,用于存储通用注释信息,分离后保留;.note.gnu.build-id节区类型为注释(NOTE)类,存储构建唯一标识,分离后保留;.dynsym节区类型为动态符号(DYNSYM)类,是动态符号表,存储运行时动态链接所需符号信息,分离后保留;.gnu.hash节区类型为GNU哈希表(GNU_HASH)类,用于快速索引动态符号,分离后保留;.hash节区类型为哈希表(HASH)类,是传统哈希表,用于动态符号快速索引,分离后保留;.dynstr节区类型为字符串表(STRTAB)类,是动态字符串表,存储动态符号对应名称字符串,分离后保留;.dynamic节区类型为动态(DYNAMIC)类,存储动态链接信息,分离后保留;.bss节区类型为无位(NOBITS)类,是未初始化数据段,运行时分配内存并初始化为0,分离后保留。

[0057] .debug_str节区类型为程序位(PROGBITS)类,存储调试字符串,属于调试相关节区,分离后剥离;.debug_loc节区类型为程序位(PROGBITS)类,存储调试位置信息,分离后剥离;.debug_abbrev节区类型为程序位(PROGBITS)类,存储调试缩写,定义调试信息编码格式,分离后剥离;.debug_info节区类型为程序位(PROGBITS)类,存储核心调试信息,包含类型、函数、变量等调试语义数据,分离后剥离;.debug_ranges节区类型为程序位(PROGBITS)类,存储调试范围,关联代码地址区间与调试元数据,分离后剥离;.debug_frame节区类型为程序位(PROGBITS)类,存储调试帧,用于栈帧回溯与调用栈调试,分离后剥离;.debug_line节区类型为程序位(PROGBITS)类,存储调试行号,建立源代码行号与机器代码地址映射,分离后剥离。

[0058] .symtab节区类型为符号表(SYMTAB)类,是静态符号表,存储完整符号信息,为兼容性,分离后仍保留;.strtab节区类型为字符串表(STRTAB)类,是静态字符串表,为静态符

号表提供字符串支撑,为兼容性,分离后仍保留;.comment节区类型为程序位(PROGBITS)类,存储编译器注释,记录编译器版本等信息,分离后保留;.shstrtab节区类型为字符串表(STRTAB)类,是节区名称表,存储所有节区名称字符串,分离后保留,用于解析文件节区结构;.topscv_kernel_desc节区类型为程序位(PROGBITS)类,存储内核描述信息,用于内核识别与设备适配,分离后保留;.gnu_debuglink节区类型为程序位(PROGBITS)类,是新增的调试链接节区,存储独立调试符号文件关联信息,分离后新增保留,用于建立运行时文件与调试符号的映射。

[0059] 表1

节区名称	类型	说明	分离后保留
.text	PROGBITS	代码段	是
.note	NOTE	通用注释	是
.note.gnu.build-id	NOTE	构建唯一标识	是
.dynsym	DYNSYM	动态符号表	是
.gnu.hash	GNU_HASH	GNU 哈希表	是
.hash	HASH	传统哈希表	是
.dynstr	STRTAB	动态字符串表	是
.dynamic	DYNAMIC	动态链接信息	是
.bss	NOBITS	未初始化数据	是
.debug_str	PROGBITS	调试字符串	否
.debug_loc	PROGBITS	调试位置信息	否
.debug_abbrev	PROGBITS	调试缩写	否
.debug_info	PROGBITS	调试信息	否
.debug_ranges	PROGBITS	调试范围	否
.debug_frame	PROGBITS	调试帧	否
.debug_line	PROGBITS	调试行号	否
.symtab	SYMTAB	静态符号表	是(兼容性保留)
.strtab	STRTAB	静态字符串表	是(兼容性保留)
.comment	PROGBITS	编译器注释	是
.shstrtab	STRTAB	节区名称表	是
.topscv_kernel_desc	PROGBITS	内核描述	是
.gnu_debuglink	PROGBITS	调试链接	是(新增)

通过将调试目标节区限定为以调试标识为前缀命名的节区,能够在剥离操作中仅移除纯粹的调试冗余数据,同时完整保留支撑异构计算设备端代码运行的全部核心数据节

区,既可以通过代码段保障程序可执行指令的正常运行,又能通过动态符号表、动态字符串表、动态链接信息及哈希表节区实现高效的动态链接与符号查找,还能借助未初始化数据段、静态符号表与静态字符串表的兼容性保留维持程序运行稳定性,配合通用注释节区、编译器注释节区、内核描述节区及构建唯一标识节区,既保留了文件标识、内核适配与版本校验能力,又避免了不必要的体积膨胀,从而在彻底分离调试信息、缩减文件体积的同时,确保轻量级可执行文件在异构计算设备上的功能完整性、运行兼容性与调试可追溯性,实现轻量化部署与高效调试的双重目标。

[0060] S140、在轻量级可执行文件中写入调试链接节区,得到运行时可执行文件。

[0061] 其中,所述调试链接节区用于使运行时可执行文件与独立调试符号文件建立关联映射。

[0062] 本发明实施例中,运行时可执行文件具体可以理解为:写入调试链接信息后的轻量化可执行文件,可直接在异构计算设备运行。

[0063] S150、将运行时可执行文件和独立调试符号文件组成完整调试目录,发送至异构计算设备。

[0064] 其中,所述运行时可执行文件用于在异构计算设备接收后加载至内存并执行,且在执行过程中响应调试请求时,基于所述调试链接节区关联加载对应的独立调试符号文件,以执行与调试请求匹配的调试操作。

[0065] 本发明实施例中,完整调试目录具体可以理解为:由运行时可执行文件与独立调试符号文件共同构成的文件集合,本质是将分离后的两类文件组织为一个可整体分发、部署的逻辑单元,便于异构计算设备统一接收与管理。

[0066] 具体的,在剥离调试目标节区得到轻量级可执行文件后,向该文件中写入用于建立文件关联的调试链接节区,从而生成可直接运行的运行时可执行文件,例如使用 `objcopy --add-gnu-debuglink` 命令向文件中写入 `.gnu_debuglink` 调试链接节区,或通过自定义节区写入方式添加调试符号文件关联信息。

[0067] 调试链接节区可以存储调试文件名、循环冗余校验(Cyclic Redundancy Check, CRC)值以及构建唯一标识等信息,以此生成运行时可执行文件。其中, `objcopy --add-gnu-debuglink` 具体可以理解为:一种向ELF文件写入调试符号文件引用的工具命令。

[0068] 将运行时可执行文件与独立调试符号文件组合成完整调试目录并发送至异构计算设备,设备接收后加载运行时可执行文件,当触发调试请求时,可根据调试链接节区中的信息自动定位、校验并加载对应的独立调试符号文件,在不影响运行效率的前提下实现调试功能的无缝衔接。

[0069] 需要强调的,在基于GNU工具链的实现方式中,通常将数据载体命名为节区(section),而本发明实施例所提出的异构计算胖二进制调试符号与运行时文件分离方法,并不局限于GNU工具链,同样能够适用于其他各类编译工具链。

[0070] 除了GNU工具链,部分编译工具链存在节区名称变更或内容独立化的处理方式,例如CUDA(Compute Unified Device Architecture,统一计算设备架构)线性代数子程序库领域特定语言组件将胖二进制节区更名为“`.lrodata(read-only data section (large/long variant),大只读数据节区)`”。其中,“`.lrodata`”是ELF文件中的一种只读数据节区。

[0071] “`Tilelang(Tile Language,图块语言)`”是一种针对GPU张量或分块计算优化的领

域特定语言,用于生成高效计算内核,通过Tilelang对内核的打包则独立为单独的kernel.cubin(kernel CUDA Binary,内核CUDA二进制文件)文件,也即CUDA内核编译后生成的独立二进制文件,包含可在GPU上执行的机器码。

[0072] 此类节区更名或节区内容独立为单独文件的形态变化,只要其本质是在二进制文件中包含可独立运行的二进制内容(包括可直接执行的可执行文件(Executable file, exe)格式或共享动态库格式),均属于本发明实施例提出的一种异构计算胖二进制调试符号与运行时文件分离的方法的保护范围。

[0073] 图2是本发明实施例适用的一种调试符号分离流程示意图,如图2所示,该调试符号分离流程从源代码(.tops,即自定义格式的源文件)开始,先通过带-g参数的编译操作生成目标文件(.o,即包含调试信息的中间目标文件),再通过带--build-id参数的链接操作生成原始ELF文件(a.out,可执行与可链接格式文件,即原始可执行文件),原始ELF文件中包含.text代码段、.symtab静态符号表、.debug_系列调试节区及.note.gnu.build-id构建唯一标识节区(Build ID,即用于唯一标识构建版本的标识);随后通过objcopy工具操作完成分离:一是执行objcopy --only-keep-debug命令,将.debug_等调试相关节区提取为独立的Debug文件(以 {build-id}.debug命名,包含完整调试信息,即独立调试符号文件),二是执行objcopy --strip-debug命令,从原始ELF文件中剥离调试节区,得到保留.symtab和.strtab以保障兼容性的轻量级运行时ELF文件(即轻量级可执行文件),三是执行objcopy --add-gnu-debuglink命令,在轻量级可执行文件中写入.gnu_debuglink调试链接节区,该节区存储调试文件名、CRC校验值及构建唯一标识,使轻量级可执行文件与独立Debug文件建立关联映射,得到运行时可执行文件,最终形成可分发至异构计算设备的完整调试目录,设备加载运行时ELF文件(即运行时可执行文件)执行程序时,可通过调试链接节区自动关联加载对应的Debug文件,实现高效运行与精准调试的解耦协同。

[0074] 图3是本发明实施例适用的一种可执行文件结构对比示意图,如图3所示,该可执行文件结构对比示意图展示了调试符号分离前后的节区变化:分离前的原始ELF文件包含.text代码段(占比 8.1%)、.note.gnu.build-id构建唯一标识节区(Build-ID,用于版本校验)、.dynsym/.dynstr动态符号表与动态字符串表(支撑动态链接)、.symtab/.strtab 静态符号表与静态字符串表、.bss未初始化数据段(占比14.1%),以及大量.debug_系列调试节区(如.debug_loc调试位置节区占比65.0%、.debug_info调试信息节区占比24.8%、.debug_str调试字符串节区占比9.1% 及其他.debug_调试节区),整体体积冗余较大;分离后的运行时ELF文件移除了全部.debug_*(即以.debug_为前缀)调试节区(调试信息被提取至独立调试符号文件),仅保留.text代码段、.note.gnu.build-id构建唯一标识节区、.dynsym/.dynstr动态符号相关节区、为兼容性保留的.symtab/.strtab静态符号相关节区、.bss未初始化数据段,同时新增.gnu_debuglink调试链接节区(用于关联独立调试符号文件),并保留.comment注释节区与.shstrtab节区名称表,在大幅缩减文件体积、提升运行效率的同时,既保障了程序运行的兼容性与功能完整性,又通过新增的调试链接节区实现了与独立调试符号文件的精准关联,兼顾了轻量化部署与调试能力。

[0075] 本发明实施例的技术方案,通过编译处理设备将原始可执行文件中的调试相关节区提取并构建独立调试符号文件,剥离原始可执行文件中无需参与运行的调试目标节区,形成轻量级的运行时可执行文件,减小运行时可执行文件的体积,降低该文件在异构计算

设备与编译处理设备之间的传输带宽占用,减少异构计算设备的存储资源消耗,且轻量级的运行时可执行文件能加快异构计算设备加载文件的速度,提升程序运行效率;在运行时可执行文件中写入用于建立关联映射的调试链接节区,使得异构计算设备在执行运行时可执行文件的过程中响应调试请求时,基于该调试链接节区关联加载对应的独立调试符号文件,无需重新传输完整的带调试符号的原始可执行文件,保证了调试操作的正常执行,实现调试能力的完整保留,降低了调试过程中的数据传输成本,将运行时可执行文件与独立调试符号文件组成完整调试目录发送,可确保二者的对应性,避免调试符号与运行时文件不匹配的问题,提升调试准确性和效率,整体适配GPU、NPU、XPU等异构计算设备的轻量化部署与高效调试需求,兼顾了程序运行性能与调试便捷性。

[0076] 实施例二

图4为本发明实施例二提供的另一种异构计算胖二进制调试符号与运行时文件分离的方法的流程图,本实施例可适用于基于分离的胖二进制调试符号与运行时文件进行异构计算的情况,该方法可以由异构计算胖二进制调试符号与运行时文件分离的装置来执行,该异构计算胖二进制调试符号与运行时文件分离的装置可以采用硬件和/或软件的形式实现,并一般可配置于异构计算设备中。

[0077] 具体的,异构计算设备与编译处理设备为两个独立的设备,编译处理设备用于完成源代码编译、链接、调试符号分离等全部预处理工作,生成轻量化运行时可执行文件与独立调试符号文件并打包发送至异构计算设备,异构计算设备则仅负责加载运行时可执行文件、执行程序,并在调试请求触发时通过内置运行时可执行文件加载器、动态链接器与调试器,关联加载对应的独立调试符号文件完成调试操作,实现编译处理与运行调试的跨设备解耦执行。

[0078] 其中,运行时可执行文件加载器具体可以理解为:用于将运行时可执行文件从存储介质读入内存,完成文件格式解析、段地址映射、内存分配等初始化工作,为程序执行准备好运行环境。动态链接器具体可以理解为:在程序加载或运行阶段,解析动态符号表与动态链接信息,完成共享库的加载、符号重定位与地址绑定,保证程序能正确调用外部库函数。调试器具体可以理解为:用于程序调试的工具软件,可通过断点、单步执行等方式控制程序流程,并依据调试链接节区关联加载独立调试符号文件,实现源码级调试、变量查看、堆栈回溯等操作。

[0079] 相应的,如图4所示,该方法包括:

S410、接收编译处理设备发送的完整调试目录,并将完整调试目录中的运行时可执行文件加载至内存中执行。

[0080] 其中,所述运行时可执行文件为剥离掉原始可执行文件中的调试目标节区后得到的轻量级可执行文件,在所述运行时可执行文件中包括使运行时可执行文件与独立调试符号文件建立关联映射的调试链接节区。

[0081] S420、在执行内存中的运行时可执行文件的过程中,响应于调试请求,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作。

[0082] 本发明实施例中,调试请求具体可以理解为:触发程序调试行为的指令或事件。

[0083] 具体的,异构计算设备通过内置的运行时可执行文件加载器接收编译处理设备下

发的完整调试目录,将其中的运行时可执行文件加载至内存,通过动态链接器完成动态符号解析、重定位与程序执行,该运行时可执行文件是剥离原始可执行文件中调试目标节区后得到的轻量级文件,且内部包含用于与独立调试符号文件建立关联映射的调试链接节区。

[0084] 在运行时可执行文件执行过程中,设备内置的调试器接收并响应调试请求,读取运行时可执行文件中的调试链接节区,并依据该节区建立的关联关系加载完整调试目录中对应的独立调试符号文件,例如在完整调试目录中,通过调试符号文件的名称与CRC循环冗余校验值进行匹配加载,或通过文件对应的构建唯一版本标识进行检索定位,或是按照预设的文件路径索引规则直接查找,从而快速加载对应的独立调试符号文件,进而完成与调试请求相匹配的调试操作。

[0085] 可选的,在上述各实施例的基础上,调试链接节区存储独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识;相应的,在上述各实施例的基础上,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作,可以包括:

读取运行时可执行文件,解析所述运行时可执行文件的调试链接节区,得到调试关联信息;其中,调试关联信息包括引用标识、循环冗余校验值及构建唯一标识;根据调试关联信息,在与引用标识匹配的搜索路径中,搜索与运行时可执行文件匹配的独立调试符号文件;若搜索到匹配的独立调试符号文件,则基于循环冗余校验值和构建唯一标识对独立调试符号文件进行校验,在校验通过后将该独立调试符号文件加载至内存,并将独立调试符号文件与内存中匹配的运行时可执行文件建立关联;基于运行时可执行文件和关联的独立调试符号文件,执行与调试请求匹配的调试操作。

[0086] 本发明实施例中,调试关联信息具体可以理解为:运行时可执行文件中,用于定位、校验、绑定独立调试符号文件的一组关键信息,是实现轻量化运行时可执行文件和外部调试符号协同工作的基础。

[0087] 具体的,调试链接节区(如.gnu_debuglink等标准格式)中存储有独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识。在执行调试操作时,调试器在加载运行时可执行文件后,解析该节区得到调试关联信息,可以包括引用标识、循环冗余校验值及构建唯一标识。其中,引用标识可以为独立调试符号文件的文件名、地址或哈希标识等。

[0088] 根据调试关联信息在对应的搜索路径中查找与运行时可执行文件匹配的独立调试符号文件,例如在运行时文件同级目录、系统专用符号目录、构建缓存目录等多条路径中并行搜索。

[0089] 若查找到对应独立调试符号文件,则通过循环冗余校验值校验数据完整性、通过构建唯一标识校验版本一致性,还可增加时间戳、签名信息等多重校验方式,校验通过后将调试符号文件加载至内存并与运行时可执行文件的地址空间建立映射关联,以此恢复断点调试、变量查看、堆栈回溯等完整调试能力,执行与调试请求匹配的调试操作。

[0090] 由于运行时文件符合标准调试器接口规范,GDB(GNU Debugger,GNU调试器)等通用调试器无需深度改造即可自动完成符号加载,同时还支持自定义搜索策略、远程符号加载、加密符号文件解析等扩展方式,适用于各类异构计算设备的调试场景。

[0091] 通过在调试链接节区同时存储引用标识、循环冗余校验值以及构建唯一标识,使

得调试器能够精准定位到对应的独立调试符号文件,再通过校验保证文件版本一致且未被篡改,能够实现调试符号与运行时文件的解耦,降低运行时文件体积与设备加载开销,又能可靠恢复完整调试能力,避免因符号不匹配或文件损坏导致调试失效,同时提升了异构计算环境下调试过程的稳定性、准确性与兼容性。

[0092] 进一步的,在上述各实施例的基础上,在搜索与运行时可执行文件匹配的独立调试符号文件之后,还可以包括:

若未搜索到匹配的独立调试符号文件,或者基于循环冗余校验值对独立调试符号文件的校验未通过,或者基于构建唯一标识对独立调试符号文件的一致性验证未通过,则基于运行时可执行文件中的调试相关节区执行与调试请求匹配的调试操作。

[0093] 具体的,在搜索与运行时可执行文件对应的独立调试符号文件之后,若因运行时可执行文件未写入调试链接节区、关联信息错误、符号文件缺失、文件损坏或版本不匹配等原因导致未搜索到匹配文件或校验验证失败,调试器会自动进入向后兼容模式,不再加载外部独立调试符号,直接读取并使用运行时可执行文件内部保留的调试相关节区信息执行调试操作,该降级机制不仅兼容传统未分离符号的运行环境,还可扩展支持自定义降级策略、日志记录、异常提示以及远程符号重试等方式,确保在异构计算设备上调试流程不中断、功能可正常使用。

[0094] 通过在未搜索到匹配的独立调试符号文件、校验值校验不通过或构建唯一标识验证不一致时,自动回退使用运行时可执行文件自身的调试相关节区完成调试,能够有效避免因外部调试符号缺失、损坏或版本不匹配导致调试功能完全失效的问题,进而提升调试流程的容错能力与稳定性,同时实现对分离符号与未分离符号两种场景的兼容,保障异构计算设备在各类异常情况下仍可正常执行调试操作,提高整体方案的适用性与可靠性。

[0095] 图5是本发明实施例适用的一种运行时兼容性加载示意图,如图5所示,加载fatbin文件并解析ELF,通过统一符号表读取接口(从.symtab读取符号、从.strtab读取字符串,且分离前后接口保持不变)完成符号处理,在分离前后对比环节中,分离前的ELF包含.symtab、.strtab及.debug_*等调试信息节区,分离后的轻量级ELF则保留.symtab与.strtab节区以确保兼容性,去除.debug_*调试信息节区,最终获取内核符号地址和入口点;该流程通过保留运行时所需节区、维持符号表接口不变,实现了对分离前后ELF的兼容加载,使得现有ELF加载器与动态链接器无需修改即可正常工作,保证运行时行为与分离前完全一致。

[0096] 图6是本发明实施例适用的一种调试器加载示意图,如图6所示,当设备内存对象加载事件触发后,调试器首先读取.note.gnu.build-id节区获取Build-ID标识XXX,再读取.gnu_debuglink节区获取调试文件名YYY,随后在可执行文件同目录、可执行文件目录下的.debug/目录及系统级/usr/lib/debug/目录等预设路径中搜索匹配的调试符号文件,最后对找到的文件进行CRC校验、构建唯一标识校验并加载调试符号;该流程通过解析Build-ID与.gnu_debuglink关联信息实现调试符号的精准定位与加载,使断点、变量查看、堆栈回溯等调试功能与未分离调试符号时保持一致,若未找到匹配文件或ELF无关联信息则自动退化为仅使用运行时文件内信息,保证了向后兼容性与调试流程的稳定性。

[0097] 本发明实施例的技术方案,通过采用经过调试目标节区剥离后的轻量级运行时可执行文件,使得异构计算设备在加载执行时能够有效减小文件体积,降低存储占用与内存

开销,提升加载速度与运行效率,通过在运行时可执行文件中设置调试链接节区,能够在接收到调试请求时,基于该节区建立的关联映射,快速定位并加载完整调试目录中对应的独立调试符号文件,既保证了程序在生产运行阶段的轻量化执行,又能在需要调试时快速恢复完整的调试信息支持,实现断点、堆栈回溯等调试操作,避免了因完全剥离调试信息导致无法调试的问题,无需在设备端长期存储庞大的完整调试信息,从而在运行性能与调试能力之间实现良好平衡,适配异构计算设备高效部署与便捷调试的双重需求。

[0098] 实施例三

图7为本发明实施例三提供的又一种异构计算胖二进制调试符号与运行时文件分离的方法的流程图,本实施例是对上述实施例中“将完整调试目录中的运行时可执行文件加载至内存中执行”的细化。如图7所示,该方法包括:

S710、接收编译处理设备发送的完整调试目录,并读取运行时可执行文件中核心数据节区,并将核心数据节区加载至内存中,执行内存中核心数据节区中的内容。

[0099] 其中,所述运行时可执行文件为剥离掉原始可执行文件中的调试目标节区后得到的轻量级可执行文件,在所述运行时可执行文件中包括使运行时可执行文件与独立调试符号文件建立关联映射的调试链接节区。

[0100] 具体的,接收编译处理设备发送的完整调试目录,读取运行时可执行文件内的核心数据节区,把这些节区加载到内存并运行程序内容,该过程完全基于系统标准的运行时可执行文件加载器与动态链接器完成,不对其进行任何修改、扩展或定制,从而实现对分离后的轻量级运行时可执行文件的原生、无侵入、全兼容加载,同时可兼容各类编译工具链、节区更名、内核独立文件等异构计算场景。

[0101] S720、在执行内存中的运行时可执行文件的过程中,响应于调试请求,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作。

[0102] 图8是本发明实施例适用的一种异构计算胖二进制调试符号与运行时文件分离示意图,如图8所示,在编译链接阶段,通过编译器扩展完成提取调试信息、剥离原文件调试节区、注入Build-ID等关联信息的操作,生成包含.gnu_debuglink与.note.gnu.build-id节区的轻量级运行时ELF,以及独立调试符号文件(.debug);在部署与运行阶段,轻量级运行时ELF通过操作系统或动态链接器实现原生兼容加载,因保留了.symtab与.strtab等运行时必需节区且接口不变,现有加载模块无需修改即可正常执行;在调试阶段,调试器扩展模块读取运行时ELF中的关联信息,解析后自动定位并加载对应的独立调试符号文件,实现动态关联与符号加载;三者协同完成了在减小运行时文件体积的同时,保持运行时兼容性与完整调试能力的目标。

[0103] 在一个具体的例子中,一种异构计算胖二进制调试符号与运行时文件分离的系统,由编译器扩展模块、运行时加载模块与调试器扩展模块构成:编译器扩展模块位于编译链接阶段,接收含调试信息的输入ELF,通过提取调试信息、剥离原文件调试节区、注入Build-ID与.gnu_debuglink关联信息的操作,输出轻量级运行时ELF与独立调试符号文件;运行时加载模块位于部署与运行阶段,利用现有操作系统与动态链接器的ELF加载能力,无需修改即可加载保留了.dynsym、.dynstr、.dynamic及.symtab、.strtab等运行时必需节区的轻量级ELF,实现天然兼容;调试器扩展模块位于调试阶段,通过符合标准规范的关联信

息(Build-ID或.gnu_debuglink),使GDB等主流调试器可自动读取运行时ELF中的关联信息,在预设路径中查找并加载对应独立调试符号文件,恢复断点、变量查看、堆栈回溯等完整调试能力,三者通过构建唯一标识与调试文件引用建立运行时文件与调试符号文件的一一对应,在减小运行时文件体积的同时保持运行时兼容性与完整调试能力。

[0104] 本发明实施例的技术方案,通过读取并加载运行时可执行文件中用于支撑程序正常运行的核心数据节区,能够在剔除冗余调试相关节区的基础上,仅将程序执行必需的代码、数据及链接相关信息加载至内存,由此减少无效数据的加载与占用,降低异构计算设备的内存消耗,提升文件加载与程序启动速度,同时保证设备端代码的稳定执行,使系统资源集中用于计算任务处理,进一步提高异构计算场景下的运行效率与资源利用率。通过在运行时可执行文件中设置调试链接节区,能够在接收到调试请求时,基于该节区建立的关联映射,快速定位并加载完整调试目录中对应的独立调试符号文件,既保证了程序在生产运行阶段的轻量化执行,又能在需要调试时快速恢复完整的调试信息支持,实现断点、堆栈回溯等调试操作,避免了因完全剥离调试信息导致无法调试的问题,无需在设备端长期存储庞大的完整调试信息,从而在运行性能与调试能力之间实现良好平衡,适配异构计算设备高效部署与便捷调试的双重需求。

[0105] 实施例四

图9为本发明实施例四提供的一种异构计算胖二进制调试符号与运行时文件分离的装置的结构示意图。如图9所示,该装置包括:文件获取模块910、独立文件模块920、轻量文件模块930、写入节区模块940和目录发送模块950,其中:

文件获取模块910,用于获取待处理的原始可执行文件;

独立文件模块920,用于在所述原始可执行文件中定位调试相关节区,并根据所述调试相关节区,构建得到独立调试符号文件;

轻量文件模块930,用于剥离原始可执行文件中调试目标节区,构建轻量级可执行文件;其中,调试相关节区的内容至少包含调试目标节区的内容;

写入节区模块940,用于在轻量级可执行文件中写入调试链接节区,得到运行时可执行文件;其中,所述调试链接节区用于使运行时可执行文件与独立调试符号文件建立关联映射;

目录发送模块950,用于将运行时可执行文件和独立调试符号文件组成完整调试目录,发送至异构计算设备;

其中,所述运行时可执行文件用于在异构计算设备接收后加载至内存并执行,且在执行过程中响应调试请求时,基于所述调试链接节区关联加载对应的独立调试符号文件,以执行与调试请求匹配的调试操作。

[0106] 本发明实施例的技术方案,通过编译处理设备将原始可执行文件中的调试相关节区提取并构建独立调试符号文件,剥离原始可执行文件中无需参与运行的调试目标节区,形成轻量级的运行时可执行文件,减小运行时可执行文件的体积,降低该文件在异构计算设备与编译处理设备之间的传输带宽占用,减少异构计算设备的存储资源消耗,且轻量级的运行时可执行文件能加快异构计算设备加载文件的速度,提升程序运行效率;在运行时可执行文件中写入用于建立关联映射的调试链接节区,使得异构计算设备在执行运行时可执行文件的过程中响应调试请求时,基于该调试链接节区关联加载对应的独立调试符号文

件,无需重新传输完整的带调试符号的原始可执行文件,保证了调试操作的正常执行,实现调试能力的完整保留,降低了调试过程中的数据传输成本,将运行时可执行文件与独立调试符号文件组成完整调试目录发送,可确保二者的对应性,避免调试符号与运行时文件不匹配的问题,提升调试准确性和效率,整体适配GPU、NPU、XPU等异构计算设备的轻量化部署与高效调试需求,兼顾了程序运行性能与调试便捷性。

[0107] 进一步的,在上述各实施例的基础上,异构计算胖二进制调试符号与运行时文件分离的装置,还可以包括:链接命令模块,其中:

链接命令模块,用于在获取待处理的原始可执行文件之前,向链接命令中注入构建唯一标识生成参数,通过执行链接命令编译链接胖二进制格式的原始可执行文件,并在原始可执行文件中写入构建唯一标识节区,得到待处理的原始可执行文件;其中,所述调试链接节区存储独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识;所述构建唯一标识用于唯一标识原始可执行文件的构建版本,使运行时可执行文件与独立调试符号文件保持版本一致。

[0108] 在上述各实施例的基础上,调试相关节区可以包括以调试标识为前缀命名的节区、用于存储静态符号信息的静态符号表节区和用于存储静态字符串信息的静态字符串表节区;所述调试链接节区存储独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识。

[0109] 在上述各实施例的基础上,调试目标节区可以包括:以调试标识为前缀命名的节区;相应的,在上述各实施例的基础上,轻量文件模块930,具体用于:

剥离原始可执行文件中调试目标节区,保留用于支撑异构计算设备端代码运行的核心数据节区,得到轻量级可执行文件;其中,所述核心数据节区包括用于存储可执行指令的代码段、用于存储通用注释信息的通用注释节区、用于存储动态符号信息的动态符号表、用于存储哈希索引的哈希表节区、用于存储动态字符串信息的动态字符串表、用于存储动态链接指令的动态链接信息、用于存储未初始化数据的未初始化数据段、用于存储静态符号信息的调试相关节区中的静态符号表节区、用于存储静态字符串信息的调试相关节区中的静态字符串表节区、用于存储编译器生成注释的编译器注释节区、用于存储节区名称映射的节区名称表节区、用于存储内核识别描述信息的内核描述节区、以及用于存储构建唯一标识的构建唯一标识节区。

[0110] 本发明实施例所提供的异构计算胖二进制调试符号与运行时文件分离的装置可执行本发明任意实施例所提供的由编译处理设备执行的异构计算胖二进制调试符号与运行时文件分离的方法,具备执行方法相应的功能模块和有益效果。

[0111] 实施例五

图10为本发明实施例五提供的另一种异构计算胖二进制调试符号与运行时文件分离的装置的结构示意图。如图10所示,该装置包括:目录接收模块1010和调试响应模块1020,其中:

目录接收模块1010,用于接收编译处理设备发送的完整调试目录,并将完整调试目录中的运行时可执行文件加载至内存中执行;

其中,所述运行时可执行文件为剥离掉原始可执行文件中的调试目标节区后得到的轻量级可执行文件,在所述运行时可执行文件中包括使运行时可执行文件与独立调试符

号文件建立关联映射的调试链接节区；

调试响应模块1020,用于在执行内存中的运行时可执行文件的过程中,响应于调试请求,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作。

[0112] 本发明实施例的技术方案,通过采用经过调试目标节区剥离后的轻量级运行时可执行文件,使得异构计算设备在加载执行时能够有效减小文件体积,降低存储占用与内存开销,提升加载速度与运行效率,通过在运行时可执行文件中设置调试链接节区,能够在接收到调试请求时,基于该节区建立的关联映射,快速定位并加载完整调试目录中对应的独立调试符号文件,既保证了程序在生产运行阶段的轻量化执行,又能在需要调试时快速恢复完整的调试信息支持,实现断点、堆栈回溯等调试操作,避免了因完全剥离调试信息导致无法调试的问题,无需在设备端长期存储庞大的完整调试信息,从而在运行性能与调试能力之间实现良好平衡,适配异构计算设备高效部署与便捷调试的双重需求。

[0113] 在上述各实施例的基础上,目录接收模块1010,具体用于:

读取运行时可执行文件中核心数据节区,并将核心数据节区加载至内存中,执行内存中核心数据节区中的内容。

[0114] 在上述各实施例的基础上,调试链接节区存储独立调试符号文件的引用标识、循环冗余校验值以及构建唯一标识;相应的,在上述各实施例的基础上,调试响应模块1020,具体用于:

读取运行时可执行文件,解析所述运行时可执行文件的调试链接节区,得到调试关联信息;其中,调试关联信息包括引用标识、循环冗余校验值及构建唯一标识;根据调试关联信息,在与引用标识匹配的搜索路径中,搜索与运行时可执行文件匹配的独立调试符号文件;若搜索到匹配的独立调试符号文件,则基于循环冗余校验值和构建唯一标识对独立调试符号文件进行校验,在校验通过后将该独立调试符号文件加载至内存,并将独立调试符号文件与内存中匹配的运行时可执行文件建立关联;基于运行时可执行文件和关联的独立调试符号文件,执行与调试请求匹配的调试操作。

[0115] 可选的,在上述各实施例的基础上,调试响应模块1020,可以包括:未通过单元,其中:

未通过单元,用于在搜索与运行时可执行文件匹配的独立调试符号文件之后,若未搜索到匹配的独立调试符号文件,或者基于循环冗余校验值对独立调试符号文件的校验未通过,或者基于构建唯一标识对独立调试符号文件的一致性验证未通过,则基于运行时可执行文件中的调试相关节区执行与调试请求匹配的调试操作。

[0116] 本发明实施例所提供的异构计算胖二进制调试符号与运行时文件分离的装置可执行本发明任意实施例所提供的由异构计算设备执行的异构计算胖二进制调试符号与运行时文件分离的方法,具备执行方法相应的功能模块和有益效果。

[0117] 本公开的技术方案中,所涉及的用户个人信息的收集、存储、使用、加工、传输、提供和公开等处理,均符合相关法律法规的规定,且不违背公序良俗。

[0118] 实施例六

图11示出了可以用来实施本发明的实施例的编译处理设备1110的结构示意图。编译处理设备旨在表示各种形式的数字计算机,诸如,膝上型计算机、台式计算机、工作台、个

人数字助理、服务器、刀片式服务器、大型计算机、和其它适合的计算机。编译处理设备还可以表示各种形式的移动装置,诸如,个人数字处理、蜂窝电话、智能电话、可穿戴设备(如头盔、眼镜、手表等)和其它类似的计算装置。本文所示的部件、它们的连接和关系、以及它们的功能仅仅作为示例,并且不意在限制本文中描述的和/或者要求的本发明的实现。

[0119] 如图11所示,编译处理设备1110包括至少一个处理器1111,以及与至少一个处理器1111通信连接的存储器,如只读存储器(ROM) 1112、随机访问存储器(RAM) 1113等,其中,存储器存储有可被至少一个处理器执行的计算机程序,处理器1111可以根据存储在只读存储器(ROM) 1112中的计算机程序或者从存储单元1118加载到随机访问存储器(RAM) 1113中的计算机程序,来执行各种适当的动作和处理。在RAM 1113中,还可存储编译处理设备1110操作所需的各种程序和数据。处理器1111、ROM 1112以及RAM 1113通过总线1114彼此相连。输入/输出(I/O)接口1115也连接至总线1114。

[0120] 编译处理设备1110中的多个部件连接至I/O接口1115,包括:输入单元1116,例如键盘、鼠标等;输出单元1117,例如各种类型的显示器、扬声器等;存储单元1118,例如磁盘、光盘等;以及通信单元1119,例如网络设备、调制解调器、无线通信收发机等。通信单元1119允许编译处理设备1110通过诸如因特网的计算机网络和/或各种电信网络与其他设备交换信息/数据。

[0121] 处理器1111可以是各种具有处理和计算能力的通用和/或专用处理组件。处理器1111的一些示例包括但不限于中央处理单元(CPU)、图形处理单元(GPU)、各种专用的人工智能(AI)计算芯片、各种运行机器学习模型算法的处理器、数字信号处理器(DSP)、以及任何适当的处理器、控制器、微控制器等。处理器1111执行上文所描述的各个方法和处理,例如由编译处理设备执行的异构计算胖二进制调试符号与运行时文件分离的方法,也即:

获取待处理的原始可执行文件;在所述原始可执行文件中定位调试相关节区,并根据所述调试相关节区,构建得到独立调试符号文件;剥离原始可执行文件中调试目标节区,构建轻量级可执行文件;其中,调试相关节区的内容至少包含调试目标节区的内容;在轻量级可执行文件中写入调试链接节区,得到运行时可执行文件;其中,所述调试链接节区用于使运行时可执行文件与独立调试符号文件建立关联映射;将运行时可执行文件和独立调试符号文件组成完整调试目录,发送至异构计算设备;其中,所述运行时可执行文件用于在异构计算设备接收后加载至内存并执行,且在执行过程中响应调试请求时,基于所述调试链接节区关联加载对应的独立调试符号文件,以执行与调试请求匹配的调试操作。

[0122] 在一些实施例中,由编译处理设备执行的异构计算胖二进制调试符号与运行时文件分离的方法可被实现为计算机程序,其被有形地包含于计算机可读存储介质,例如存储单元1118。在一些实施例中,计算机程序的部分或者全部可以经由ROM 1112和/或通信单元1119而被载入和/或安装到编译处理设备1110上。当计算机程序加载到RAM 1113并由处理器1111执行时,可以执行上文描述的由编译处理设备执行的异构计算胖二进制调试符号与运行时文件分离的方法的一个或多个步骤。备选地,在其他实施例中,处理器1111可以通过其他任何适当的方式(例如,借助于固件)而被配置为执行由编译处理设备执行的异构计算胖二进制调试符号与运行时文件分离的方法。

[0123] 实施例七

图12示出了可以用来实施本发明的实施例的异构计算设备1210的结构示意图。异

构计算设备旨在表示各种形式的数字计算机,诸如,膝上型计算机、台式计算机、工作台、个人数字助理、服务器、刀片式服务器、大型计算机、和其它适合的计算机。异构计算设备还可以表示各种形式的移动装置,诸如,个人数字处理、蜂窝电话、智能电话、可穿戴设备(如头盔、眼镜、手表等)和其它类似的计算装置。本文所示的部件、它们的连接和关系、以及它们的功能仅仅作为示例,并且不意在限制本文中描述的和/或者要求的本发明的实现。

[0124] 如图12所示,异构计算设备1210包括至少一个处理器1211,以及与至少一个处理器1211通信连接的存储器,如只读存储器(ROM) 1212、随机访问存储器(RAM) 1213等,其中,存储器存储有可被至少一个处理器执行的计算机程序,处理器1211可以根据存储在只读存储器(ROM) 1212中的计算机程序或者从存储单元1218加载到随机访问存储器(RAM) 1213中的计算机程序,来执行各种适当的动作和处理。在RAM 1213中,还可存储异构计算设备1210操作所需的各种程序和数据。处理器1211、ROM 1212以及RAM 1213通过总线1214彼此相连。输入/输出(I/O)接口1215也连接至总线1214。

[0125] 异构计算设备1210中的多个部件连接至I/O接口1215,包括:输入单元1216,例如键盘、鼠标等;输出单元1217,例如各种类型的显示器、扬声器等;存储单元1218,例如磁盘、光盘等;以及通信单元1219,例如网络设备、调制解调器、无线通信收发机等。通信单元1219允许异构计算设备1210通过诸如因特网的计算机网络和/或各种电信网络与其他设备交换信息/数据。

[0126] 处理器1211可以是各种具有处理和计算能力的通用和/或专用处理组件。处理器1211的一些示例包括但不限于中央处理单元(CPU)、图形处理单元(GPU)、各种专用的人工智能(AI)计算芯片、各种运行机器学习模型算法的处理器、数字信号处理器(DSP)、以及任何适当的处理器、控制器、微控制器等。处理器1211执行上文所描述的各个方法和处理,例如由异构计算设备执行的异构计算胖二进制调试符号与运行时文件分离的方法,也即:

接收编译处理设备发送的完整调试目录,并将完整调试目录中的运行时可执行文件加载至内存中执行;其中,所述运行时可执行文件为剥离掉原始可执行文件中的调试目标节区后得到的轻量级可执行文件,在所述运行时可执行文件中包括使运行时可执行文件与独立调试符号文件建立关联映射的调试链接节区;在执行内存中的运行时可执行文件的过程中,响应于调试请求,读取运行时可执行文件中的调试链接节区,并基于调试链接节区,关联加载完整调试目录中对应的独立调试符号文件,执行与调试请求匹配的调试操作。

[0127] 在一些实施例中,由异构计算设备执行的异构计算胖二进制调试符号与运行时文件分离的方法可被实现为计算机程序,其被有形地包含于计算机可读存储介质,例如存储单元1218。在一些实施例中,计算机程序的部分或者全部可以经由ROM 1212和/或通信单元1219而被载入和/或安装到异构计算设备1210上。当计算机程序加载到RAM 1213并由处理器1211执行时,可以执行上文描述的由异构计算设备执行的异构计算胖二进制调试符号与运行时文件分离的方法的一个或多个步骤。备选地,在其他实施例中,处理器1211可以通过其他任何适当的方式(例如,借助于固件)而被配置为执行由异构计算设备执行的异构计算胖二进制调试符号与运行时文件分离的方法。

[0128] 本文中以上描述的系统和技术各种实施方式可以在数字电子电路系统、集成电路系统、现场可编程门阵列(FPGA)、专用集成电路(ASIC)、专用标准产品(ASSP)、芯片上系统(SOC)、负载可编程逻辑设备(CPLD)、计算机硬件、固件、软件、和/或它们的组合中

实现。这些各种实施方式可以包括：实施在一个或者多个计算机程序中，该一个或者多个计算机程序可在包括至少一个可编程处理器的可编程系统上执行和/或解释，该可编程处理器可以是专用或者通用可编程处理器，可以从存储系统、至少一个输入装置、和至少一个输出装置接收数据和指令，并且将数据和指令传输至该存储系统、该至少一个输入装置、和该至少一个输出装置。

[0129] 用于实施本发明的方法的计算机程序可以采用一个或多个编程语言的任何组合来编写。这些计算机程序可以提供给通用计算机、专用计算机或其他可编程数据处理装置的处理器，使得计算机程序当由处理器执行时使流程图和/或框图中所规定的功能/操作被实施。计算机程序可以完全在机器上执行、部分地在机器上执行，作为独立软件包部分地在机器上执行且部分地在远程机器上执行或完全在远程机器或服务器上执行。

[0130] 在本发明的上下文中，计算机可读存储介质可以是有形的介质，其可以包含或存储以供指令执行系统、装置或设备使用或与指令执行系统、装置或设备结合地使用的计算机程序。计算机可读存储介质可以包括但不限于电子的、磁性的、光学的、电磁的、红外的、或半导体系统、装置或设备，或者上述内容的任何合适组合。备选地，计算机可读存储介质可以是机器可读信号介质。机器可读存储介质的更具体示例会包括基于一个或多个线的电气连接、便携式计算机盘、硬盘、随机存取存储器 (RAM)、只读存储器 (ROM)、可擦除可编程只读存储器 (EPROM 或快闪存储器)、光纤、便捷式紧凑盘只读存储器 (CD-ROM)、光学储存设备、磁储存设备、或上述内容的任何合适组合。

[0131] 为了提供与用户的交互，可以在编译处理设备或异构计算设备上实施此处描述的系统和技术，该编译处理设备或异构计算设备具有：用于向用户显示信息的显示装置（例如，CRT（阴极射线管）或者LCD（液晶显示器）监视器）；以及键盘和指向装置（例如，鼠标或者轨迹球），用户可以通过该键盘和该指向装置来将输入提供给编译处理设备或异构计算设备。其它种类的装置还可以用于提供与用户的交互；例如，提供给用户的反馈可以是任何形式的传感反馈（例如，视觉反馈、听觉反馈、或者触觉反馈）；并且可以用任何形式（包括声输入、语音输入或者、触觉输入）来接收来自用户的输入。

[0132] 可以将此处描述的系统和技术实施在包括后台部件的计算系统（例如，作为数据服务器）、或者包括中间件部件的计算系统（例如，应用服务器）、或者包括前端部件的计算系统（例如，具有图形用户界面或者网络浏览器的用户计算机，用户可以通过该图形用户界面或者该网络浏览器来与此处描述的系统和技术实施方式交互）、或者包括这种后台部件、中间件部件、或者前端部件的任何组合的计算系统中。可以通过任何形式或者介质的数字数据通信（例如，通信网络）来将系统的部件相互连接。通信网络的示例包括：局域网 (LAN)、广域网 (WAN)、区块链网络和互联网。

[0133] 计算系统可以包括客户端和服务器。客户端和服务器一般远离彼此并且通常通过通信网络进行交互。通过在相应的计算机上运行并且彼此具有客户端-服务器关系的计算机程序来产生客户端和服务器的关系。服务器可以是云服务器，又称为云计算服务器或云主机，是云计算服务体系中的一项主机产品，以解决了传统物理主机与VPS服务中，存在的管理难度大，业务扩展性弱的缺陷。

[0134] 应该理解，可以使用上面所示的各种形式的流程，重新排序、增加或删除步骤。例如，本发明中记载的各步骤可以并行地执行也可以顺序地执行也可以不同的次序执行，只

要能够实现本发明的技术方案所期望的结果,本文在此不进行限制。

[0135] 上述具体实施方式,并不构成对本发明保护范围的限制。本领域技术人员应该明白的是,根据设计要求和因素,可以进行各种修改、组合、子组合和替代。任何在本发明的精神和原则之内所作的修改、等同替换和改进等,均应包含在本发明保护范围之内。

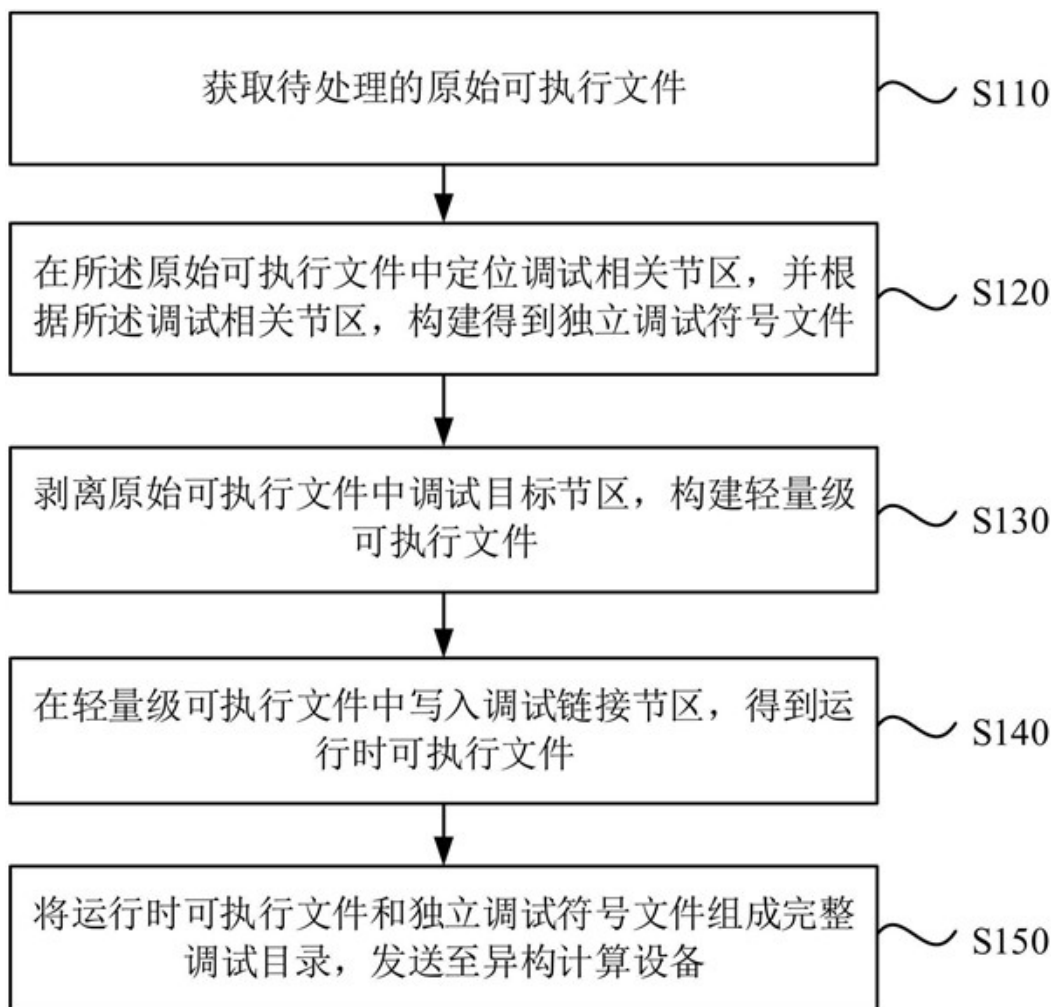


图1

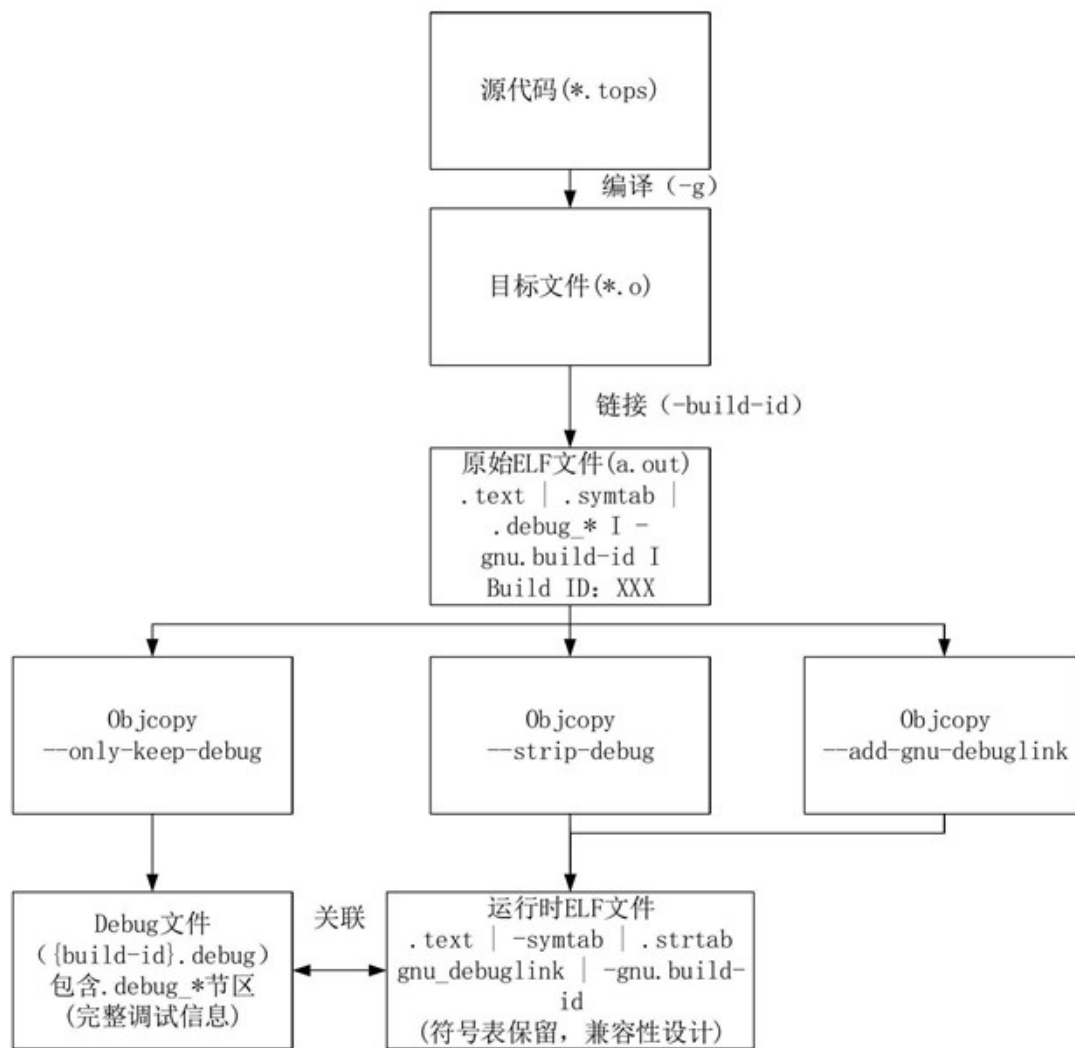


图2

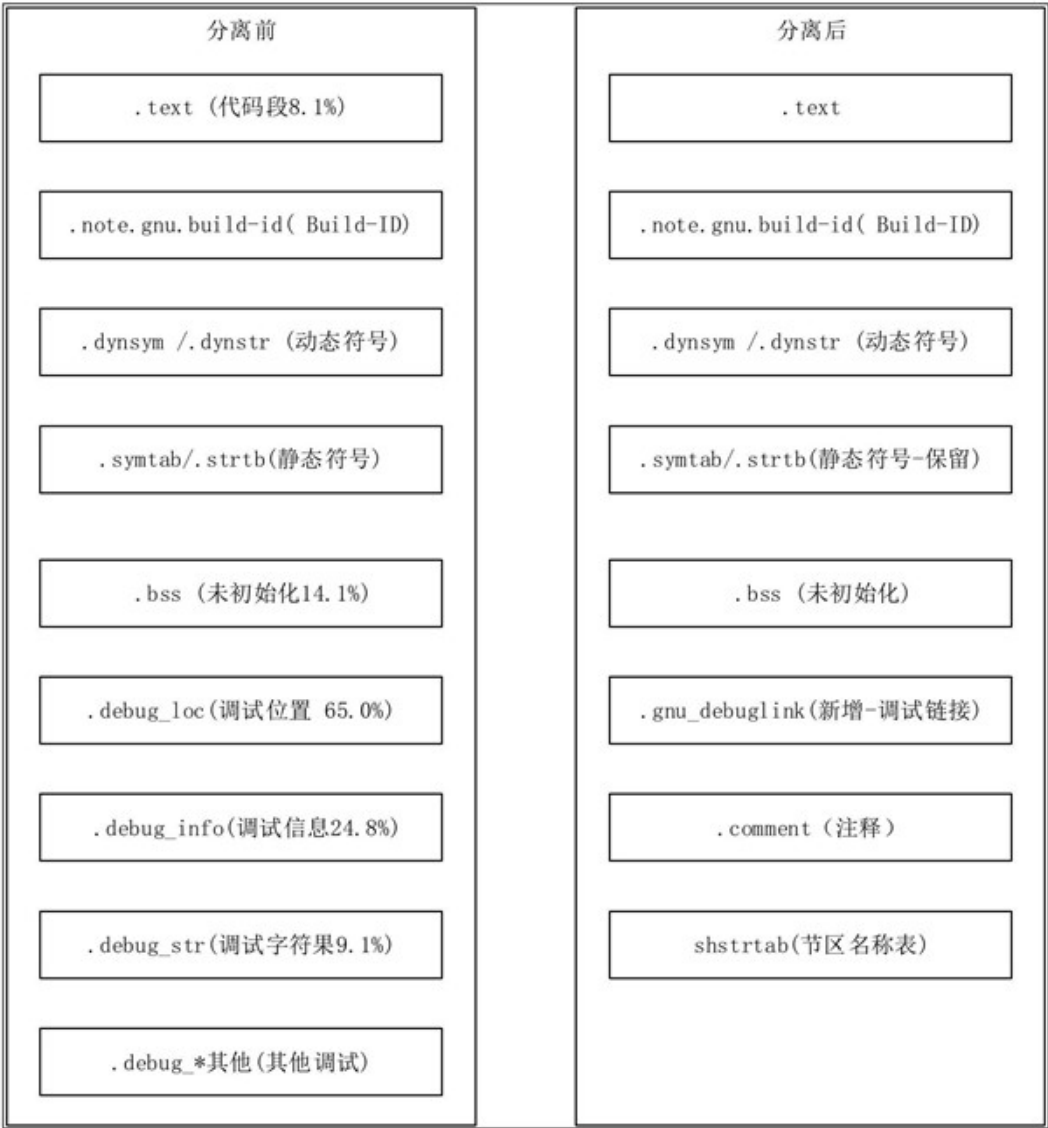


图3

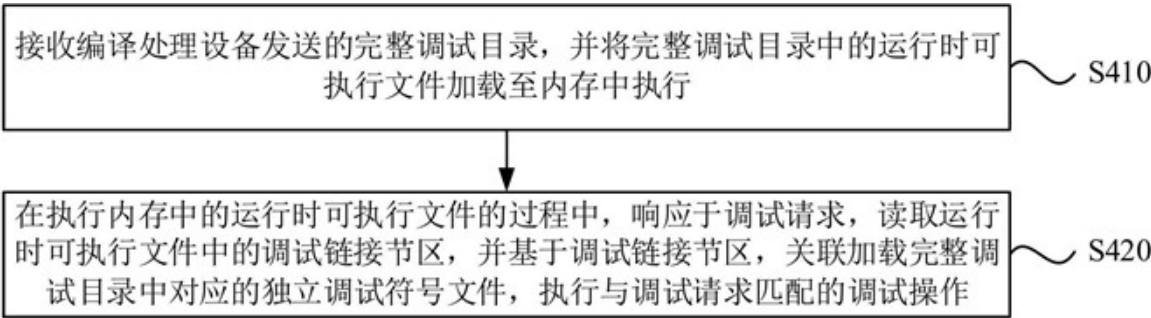


图4

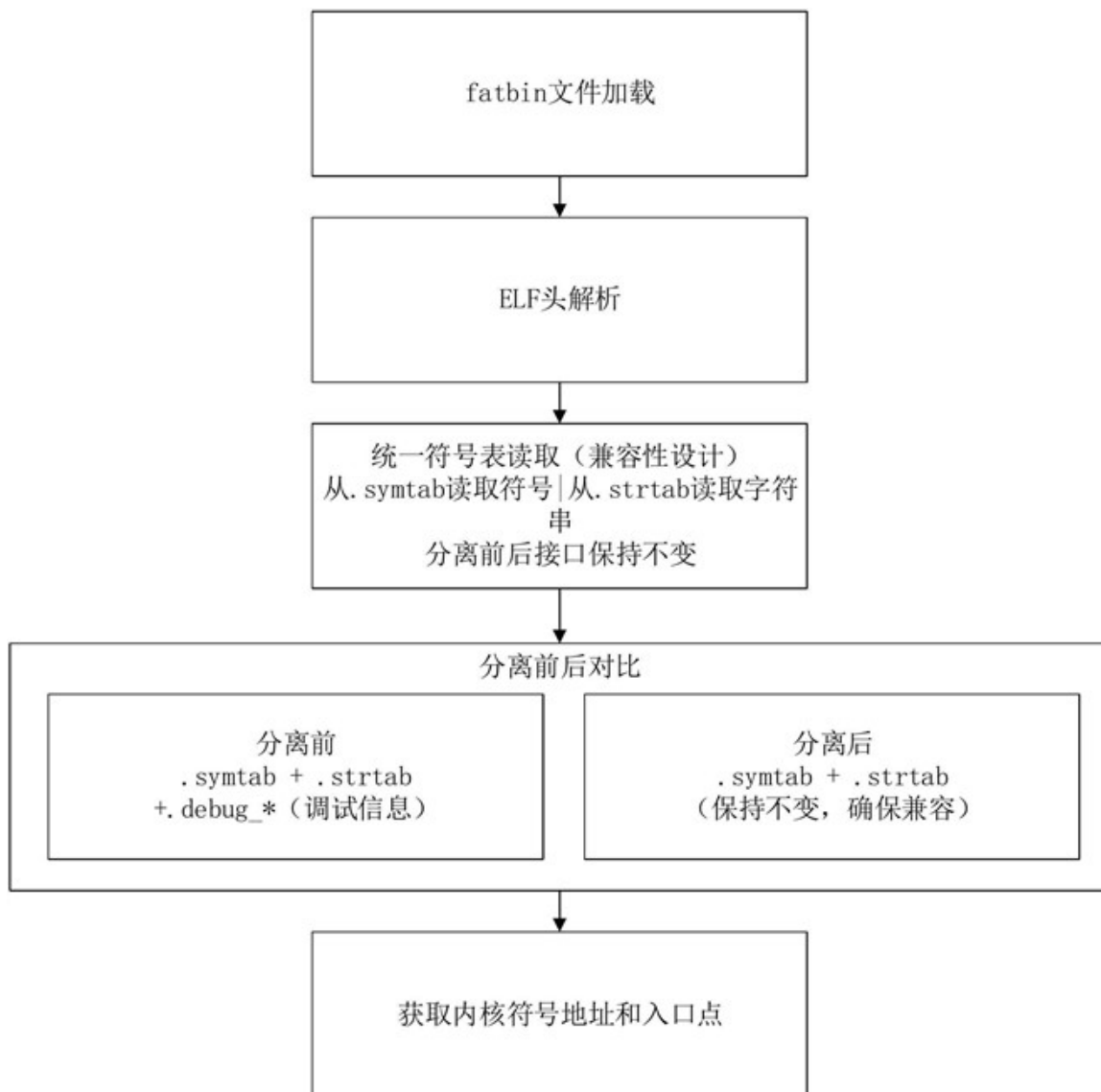


图5

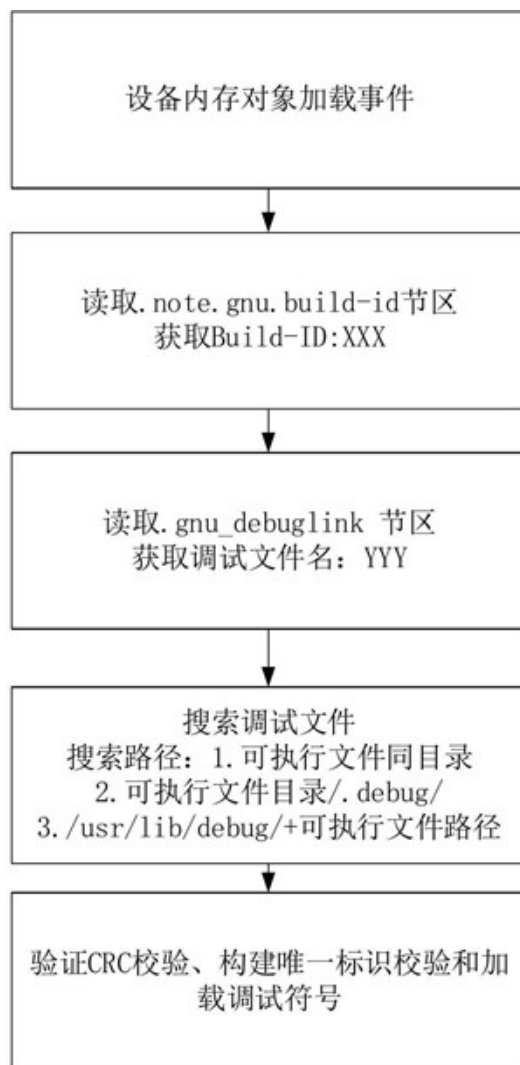


图6

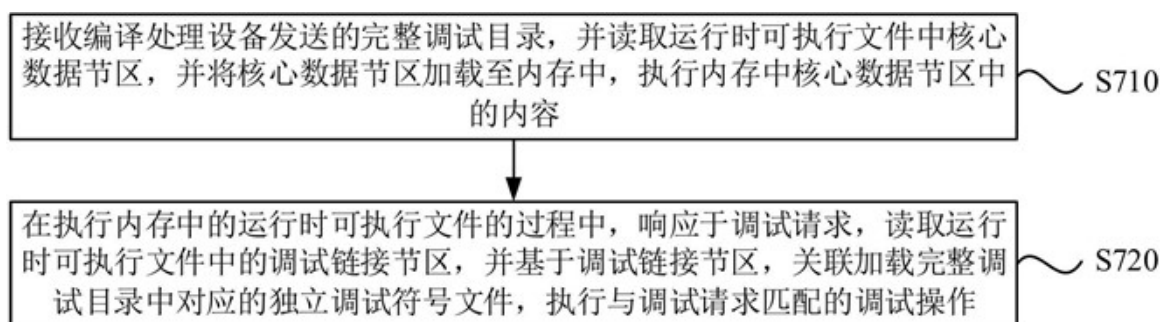


图7



图8

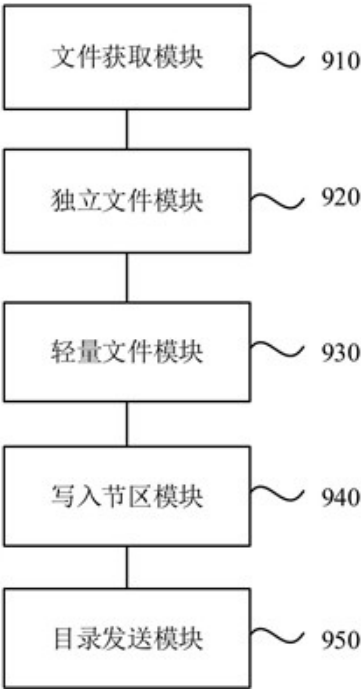


图9

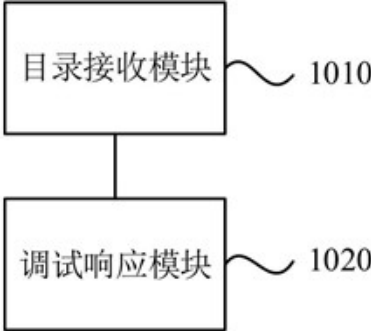


图10

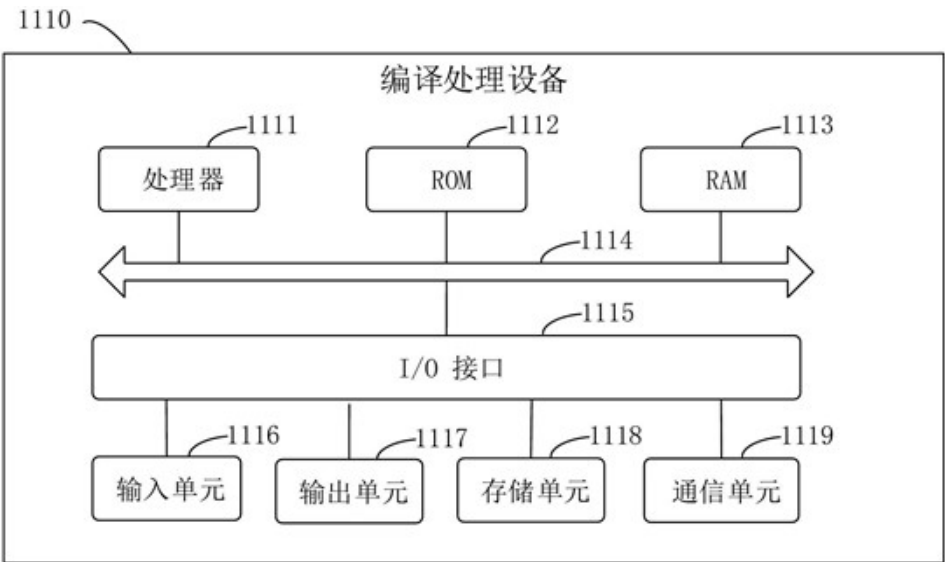


图11

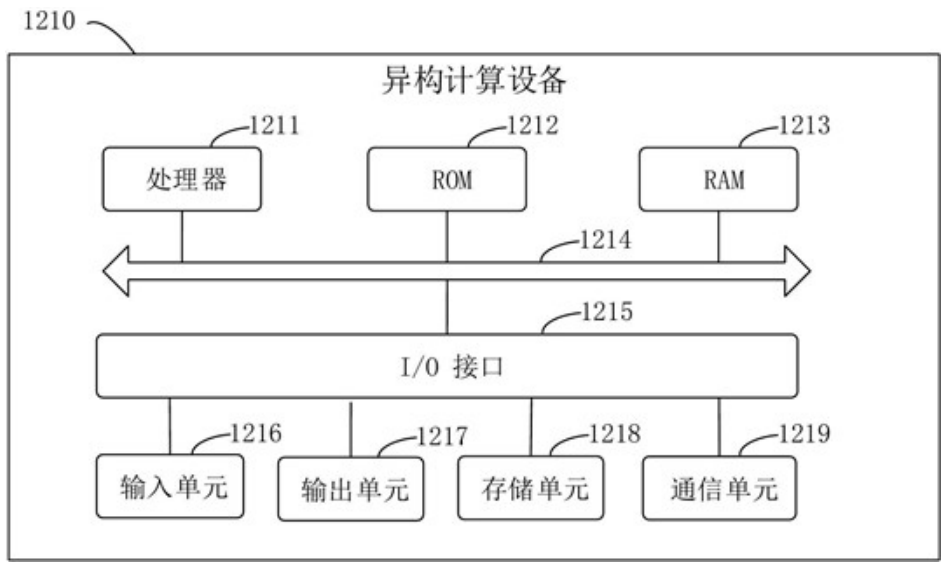


图12